

字符串与字符函数

内容取自：《C Primer Plus中文版》
•11章 字符串



四、字符数组（字符串）

- 假设有一个字符串“first”，能否用一个变量存放？
- 在C语言中没有专门的字符串变量。通常用一个字符数组来存放一个字符串，并以字符‘\0’作为字符串的结束符。‘\0’对应的ASCII码为0。

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
f	i	r	s	t	\0

- 在声明一个容纳字符串的字符数组时，数组的大小应足以容纳字符串中的字符以及字符串结束符‘\0’。

➤ 字符数组初始化 三种方式：

```
char a[256]={ 'f' , 'i' , 'r' ,  
              's' , 't' , '\0' };
```

也可写为： `char a[256]="first";`

或去掉 {} 写为： `char a[256]="first";`

内存中存放形式为：

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
f	i	r	s	t	\0

后两种初始化情况中， ‘\0’ 是系统自动加上的。

上述声明中可以不指定字符数组的长度，编译器会根据字符串的长度来确定数组的长度（字符串长度+1）。

字符数组的特点总结：

- 1、字符数组用来存放字符串，以'\0' 作为字符串的结束字符。
- 2、由于'\0'，字符数组实际能存放的其他字符个数比数组长度少1；而且在处理字符数组时，一般不用数组实际长度，而是用'\0' 来判断字符串是否结束。
- 3、对字符数组输入输出时，一般不用循环语句逐个字符地进行输入输出，而是用字符串输入输出函数整体输入输出；
- 4、字符串修改时，如果是逐个字符地赋值，当赋值结束时，程序必须将'\0'赋值给字符数组末尾，否则后续读取或打印时会因找不到结束标志而越界访问。

scanf("%s") 只能读一个单词

```
#include<stdio.h>
#define STLEN 256
int main()
{
    char st[STLEN];
    printf("input string:\n");
    scanf("%200s",st);
    printf("%s",st);
}
```

scanf函数读取用户键入的字符到字符数组，直到遇到空格、回车、或文件结束符为止。空格、回车、或文件结束符被丢弃，最后一个字符读入后往字符数组中写入结束符‘\0’。

注意：scanf不关心字符数组的大小，所以它往数组中写字符时，所写入的字符可能超出数组的范围。程序员必须自己控制。

6.3.4 字符数组

遇空格结束读取

input string:

aaaaa aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa ✓
aaaaa 请按任意键继续...

'a'	'a'	'a'	'a'	'a'	'\0'	
-----	-----	-----	-----	-----	------	-------

遇回车结束读取

input string:

aaa
a
aaaaaaaaaaaaa
aaa

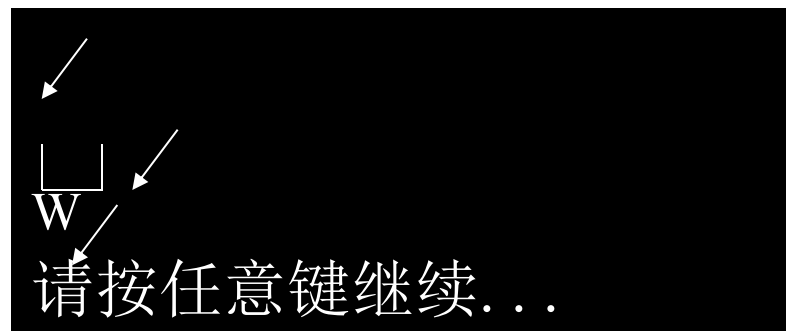
printf(“%s”)可输出任何字符串

```
#include<stdio.h>
#define STLEN 256
int main()
{
    char st[STLEN]="one string\nHa ha.\n";
    printf("%s",st);
}
```

printf输出字符串时，会将字符数组中所有内容输出，直到‘\0’，但‘\0’不输出。

scanf("%s")无法实现空行的读取

```
#include<stdio.h>
#define STLEN 201
int main()
{
    char str[STLEN];
    scanf("%s",str);
    if (str[0]=='\n')
        printf("is enter");
    else if (str[0]=='\0')
        printf("is null");
    return 0;
}
```



使用scanf，若只是输入回车或者空格，则程序一直在等待，直到输入一个非回车、非空格字符。

即：scanf无法实现空行的读取！

gets()读取一行字符串

常用的文本读取是按行进行字符串读取、打印。

- `char * gets(char * str);`

`gets()`函数简单易用，它**读取整行输入**，直至遇到换行符，然后**丢弃换行符**，存储其余字符，并在这些字符的末尾添加一个空字符使其成为一个C字符串。它经常和`puts()`函数配对使用，该函数用于显示字符串，并在末尾添加换行符。

成功则返回所读字符串指针，失败返回空指针。

- `int puts(const char * str);`

将`str`所指向的字符串输出到文件或显示器上，字符串结束标记‘\0’不会被输出。

成功则返回字符长度，失败返回-1。

gets(char *)使用示例

```
#include<stdio.h>
#include<string.h> //strlen()计算串长度
#define STLEN 201
int main()
{
    char st[STLEN];
    printf("input string:\n");
    gets(st);
    puts(st);
    printf("string len=%d\n",strlen(st));
    return 0;
}
```

测试：可读取空行、多单词一行,丢弃\n。

puts(char *)可输出包含多行的字符串

```
#include<stdio.h>
#define STLEN 201
int main()
{
    char st[STLEN]="one string\nHa ha.\n";
    puts(st);
}
```

puts输出字符串时，会将字符数组中所有内容输出，直到‘\0’，但会在末尾自动追加输出‘\n’。

- 因为不安全，gets() 从C11标准中被废除了！
- 问题出在gets()唯一的参数是字符串地址，它无法检查数组是否装得下输入行。
- 如果输入的字符串过长，会导致缓冲区溢出（buffer overflow），即多余的字符超出了指定的目标空间。如果这些多余的字符只是占用了尚未使用的内存，就不会立即出现问题；如果它们擦写掉程序中的其他数据，会导致程序异常中止；或者还有其他情况。

gets(char *)溢出示例

```
#include<stdio.h>
#include<string.h>
#define STLEN 6
int main()
{
    int a[10]={1,2,3,4,5,6,7,8,9,10};
    char st[STLEN];
    printf("input string:\n");
    gets(st);
    puts(st);
    printf("string len=%d\n",strlen(st));
    for (int i=0;i<10;i++)
        printf("%d\t",a[i]);
    int gap=(char *)a-st;
    printf("\n a=%p,st=%p,gap=%d\n",&a,&st,gap);
    return 0;
}
```

fgets()从文件读取一行字符串

fgets()函数通过第2个参数限制读入的字符数来解决溢出的问题。该函数专门设计用于处理文件输入，所以仅从键盘读串时会觉得参数太多。

- **char * fgets(char *str, int n, FILE *stream);**
从文件或键盘读取字符到str所指向的数组中，直到读够n-1个字符，或读到换行符'\n'，但**不会丢弃换行符**，或者读到文件末尾。最后一个字符读入后自动写入一个'\0'。若成功则返回str，若无字符读入数组或者读取失败返回空指针NULL。
- **int fputs(const char * str, FILE *stream);**
将str所指向的字符串输出到文件或显示器上，字符串结束标记'\0'不会被输出。

`fgets(char *,len,stdin)`三个参数使用

```
#include<stdio.h>
#define STLEN 202
int main()
{
    char st[STLEN];
    printf("input string:\n");
    fgets(st,STLEN-1,stdin);
    fputs(st,stdout);

    return 0;
}
```

`fgets()`不丢弃换行符，好处是若串太长超出数组长度，通过检查读入的`st`是否有`\n`就可知道读取不完整。

检查读取一行字符串是否所读不完整

```
int isLine(char * str)
{
    int i,line;
    for(i=0,line=0; str[i]!='\0' && !line; i++){
        if (str[i]=='\n')
            line=1;
    }
    return line;//返回0是不完整，返回1则是整行
}
```



```
#include<stdio.h>
#define STLEN 12
int isLine(char * str);
int main(){
    char st[STLEN];
    printf("input string:\n");
    fgets(st,STLEN,stdin);
    printf("string len=%d\n",strlen(st));
    fputs(st,stdout);
    if (isLine(st)==0)
        puts("read string not complete.");
    return 0;
}
//测试输入: 1234567890 hi words.
```

若行串完整则丢弃换行符

```
int isLineCutN(char * str)
{
    int i,line;
    for(i=0,line=0; str[i]!='\0' && !line; i++){
        if (str[i]=='\n'){
            str[i]='\0';
            line=1;
        }
    }
    return line;//返回0为不完整，返回1则丢弃换行符
}
```

```
#include<stdio.h>
#define STLEN 12
int isLineCutN(char * str);
int main()
{
    char st[STLEN];
    printf("input string:\n");
    fgets(st,STLEN-1,stdin);
    printf("orig len=%d\n",strlen(st));
    if (isLineCutN(st)==0)
        puts("read string not complete.");
    else
        puts("removed string tail \\n.");
    printf("now len=%d\n",strlen(st));
    fputs(st,stdout);
}
//测试输入1: 1234567890 hi words.
//测试输入2: words
```

<string.h>中常用库函数

```
size_t strlen(const char * str);
```

strlen是测试字符串长度的函数，返回值为字符串的实际长度(不包括'\0'在内)。

```
char * strcat(char *str1, const char *str2);
```

将串str2拼接到str1串末尾，返回str1指针。

```
char *strncat(char *str1, const char *str2,  
size_t count);
```

将串str2中拼接到str1串末尾，限定拼接字符数不超过count，返回str1指针。

```
#include<stdio.h>
#include<string.h>
#define SIZE 82
int isLineCutN(char * str);
int main(void)
{
    char flower[SIZE];
    char addon[]="s smell like old shoes.";
    puts("What is your favorite flower?");
    fgets(flower,SIZE,stdin);
    isLineCutN(flower);
    strcat(flower,addon);
    puts(flower);
    return 0;
}
```

安全的strncat()示例

```
#include<stdio.h>
#include<string.h>
#define SIZE 12
int isLineCutN(char * str);
int main(void)
{
    char flower[SIZE];
    char addon[]="s smell like old shoes.";
    puts("What is your favorite flower?");
    fgets(flower,SIZE,stdin);
    isLineCutN(flower);
    int maxn=SIZE-1-strlen(flower);
    strncat(flower,addon,maxn);
    puts(flower);
    return 0;
}
```

<string.h>中常用库函数

`char * strcpy(char * to, const char * from);`

将串from整个复制到to数组，返回复制字符的首地址。

例: `char str1[100], str2[100] = "Have a rest.";`

`strcpy(str1, str2);` //不能`str1 = str2`复制串

`char * strncpy(char * to, const char * from, size_t count);`

将串from复制到to数组，**限定复制字符数不超过count**，返回复制字符的首地址。注意：如果拷贝到第n个字符时还未拷贝完整个源字符串，就不会拷贝'\0'空字符。

字符串拷贝函数分析示例

```
#include<string.h>
#define SIZE 42
#define WORDS "beast"
int main(void){
    const char * orig= WORDS;
    char copy[SIZE] = "Be the best that you can be.";
    char * ps;
    puts(orig);    puts(copy);
    ps = strcpy( copy + 7, orig);
    puts(copy);//查看copy结果
    puts(ps);//ps不是copy的首地址
    return 0;
}
```


strncpy()示例(1)

```
#include<stdio.h>
#include<string.h>
#define SIZE 40
#define TARGSIZE 7
#define LIM 5
int isLineCutN(char * str);
int main(void){
    char qwords[LIM][TARGSIZE];
    char temp[SIZE];
    int i = 0;
    printf("Enter %d words beginning with q:\n", LIM);
    fgets(temp,SIZE,stdin);      isLineCutN(temp);
```

strncpy()示例(2)

```
while (i < LIM && temp[0]!='\0'){ //空行表示输入结束
    if (temp[0] != 'q')
        printf("%s doesn't begin with q!\n", temp);
    else{
        strncpy(qwords[i], temp, TARGSIZE-1);
        qwords[i][TARGSIZE-1]='\0';
        i++;
    }
    fgets(temp,SIZE,stdin);      isLineCutN(temp);
}
puts("Here are the words accepted:");
for (i = 0; i < LIM; i++)
    puts(qwords[i]);
return 0;
```

<string.h>中常用库函数

```
int strcmp(const char * str1, const char * str2);
```

比较字符串1和串2的字典序排序；若str1排在前，返回负数；
若str1==str2则返回0；若str1排在后，返回正数。

```
int strncmp(const char * str1, const char * str2,  
size_t count);
```

比较字符串1和串2 前count字符的字典序；若str1子串排在前，返回负数；若两串前count字符都相等则返回0；若str1子串排在后，返回正数。

字符串比较函数示例1

```
#include<stdio.h>
#include<string.h>
#define SIZE 12
#define ANSWER "Grant"
int isLineCutN(char * str);
int main(void){
    char try[SIZE];
    puts("Who is buried in Grant's tomb?");
    fgets(try,SIZE,stdin);      isLineCutN(try);
    while (strcmp(try,ANSWER) != 0){
        puts("No,that's wrong. Try again.");
        fgets(try,SIZE,stdin);      isLineCutN(try);
    }
    puts("That's right!");
    return 0;
```



字符串比较函数示例2

```
#include <stdio.h>
#include <string.h>
#define SIZE 202
int main(){
    char str1[SIZE],str2[SIZE];
    printf("input string1:\n");
    fgets(str1,SIZE,stdin);
    printf("reinput string2:\n");
    fgets(str2,SIZE,stdin);
    if (strcmp(str1,str2)==0)
        printf("be same.");
    else if (strcmp(str1,str2)>0)
        printf("%s > %s",str1,str2);
    else
        printf("%s < %s",str1,str2);
    return 0;
}
```

strncmp()前n字符比较示例

```
#include <stdio.h>
#include <string.h>
#define LISTSIZE 6
int main(){
    const char * list[LISTSIZE]={"astronomy", "astounding",
    "astrophysics", "ostracize", "asterism", "astrophobia"};
    int count = 0;
    int i;
    for (i = 0; i < LISTSIZE; i++)
        if (strncmp(list[i], "astro", 5) == 0){
            printf("Found: %s\n", list[i]);
            count++;
        }
    printf("The list contained %d words beginning " " with
    astro.\n", count);
    return 0;
}
```

```
int stringcompare(char s1[],char s2[]){
    int i;
    for (i=0; s1[i]!='\0' && s2[i]!='\0' &&
s1[i]==s2[i] ;i++)
        ;
    if (s1[i]=='\0' && s2[i]=='\0'){
        return 0;
    } else if (s1[i]=='\0')
        return 0-s2[i];
    else if (s2[i]=='\0')
        return s1[i]-0;
    else return (s1[i]-s2[i]);
}
```

在文本中寻找某个模式出现的位置称为字符串匹配.

`<string.h>`中常用库函数

```
char *strstr( const char *str1, const char  
*str2 );
```

串匹配函数，返回一个指针，指向字符串`str2` 首次出现于字符串`str1`中的位置，如果没有找到，返回`NULL`。


```
#include <stdio.h>
#include <string.h>
#define SIZE 256
int isLineCutN(char *);
int main(){
    char s1[SIZE],s2[SIZE];
    int loc;
    fgets(s1,SIZE,stdin); isLineCutN(s1);
    fgets(s2,SIZE,stdin); isLineCutN(s2);
    char * pm = strstr(s1,s2);
    if (pm!=NULL){
        puts(pm);
        int loc=pm-s1;
        printf("match loc=%d\n",loc);
    } else puts("Not found.");
    return 0;
}
```



- 在文本中寻找某个模式出现的位置称为字符串匹配. 我们介绍一种蛮力算法, 解决字符串匹配问题, 也叫朴素字符串匹配器.

```
procedure string match ( $n, m$ : positive integers,  
     $m \leq n, t_1, t_2, \dots, t_n, p_1, p_2, \dots, p_m$ : characters)  
    for  $s := 0$  to  $n - m$   
         $j := 1$   
        while ( $j \leq m$  and  $t_{s+j} = p_j$ )  
             $j := j + 1$   
        if  $j > m$  then print “s is a valid shift”
```

```
char * match(char * text, char * pattern,int n, int m){  
    char * ps;  
    for(ps=text; *ps!='\0'; ps++){  
        if (strncmp(ps,pattern,m)==0)  
            return ps;  
    }  
    return NULL;  
}
```

```
char * match(char t[], char p[],int n, int m){
    int s,j;
    for(s=0; s<n-m; s++){ //t[j]!='\0'
        for (j=0; s1[s+j]!='\0' && s2[j]!='\0' &&
s1[s+j]==s2[j] ;j++)
            ;
        if (j>=m)
            return &s1[s];
    }
    return NULL;
}
```

练习：输入一行字符串，统计其中有多少个单词，单词之间用空格分隔开，但允许多个空格。

- 思路：可用scanf函数特点来逐词读取；或者跟踪判断空格来统计。
- 难点：多空格如何避免重复计数？
 - 跟踪 从 字母到空格或换行符 的状态变化
 - 每次变化肯定对应一个新词

```
#include<stdio.h>
#define SIZE 256
int main(){
    char str[SIZE],ch;
    puts("input string:");
    int count=0;
    do{
        scanf("%s",str);    count++;
    }while((ch=getchar())!='\n');
    printf("word num=%d\n",count);
    return 0;
}
```

缺点：至少输入一个单词

```
int wordsNum(char str[]){
    int i,num,flag;
    for (i=0,num=0,flag=0; str[i]!='\0'; i++){
        if (str[i]!=' ' && str[i]!='\n')
            flag=1;
        else if (flag==1){
            num++; flag=0;
        }
    }
    return num;
}
```

```
#include <stdio.h>
#include <string.h>
#define SIZE 256
int wordsNum(char str[]);
int main(){
    char string[SIZE];
    int num=0;
    fgets(string,SIZE,stdin);
    num=wordsNum(string);//string末尾\n不要去掉
    printf("the words is %d\n",num);
    return 0;
}
```



```
#include<stdio.h>
#define SIZE 256
int isLineCutN(char *);
void reverseStr(char s[]);
int main(void)
{
    char string[SIZE];

    fgets(string,SIZE,stdin);
    isLineCutN(string);
    reverseStr(string);
    fputs(string,stdout);

    return 0;
}
```

```
void reverseStr(char s[]){//将字符元素逆序
    int len,i;
    char ch;
    len = strlen(s);
    for(i=0;i<=len/2-1;i++){
        ch=s[i];
        s[i]=s[len-1-i];
        s[len-1-i]=ch;
    }
}
```

